

Laboratory 3: Debugging

Google Doc Link:

https://docs.google.com/document/d/1Pb0_BLeadKitFY6BAXdc0RiPgpYwsOBb/edit?usp=sharing&ouid=110924416719706935733&rtpof=true&sd=true

Objectives

1. Understand the common error messages in Vivado.
2. Be able to use digital simulation to debug the design.
3. Be able to use ILA and VIO to debug the design.

Part 1: Dealing with Error Messages

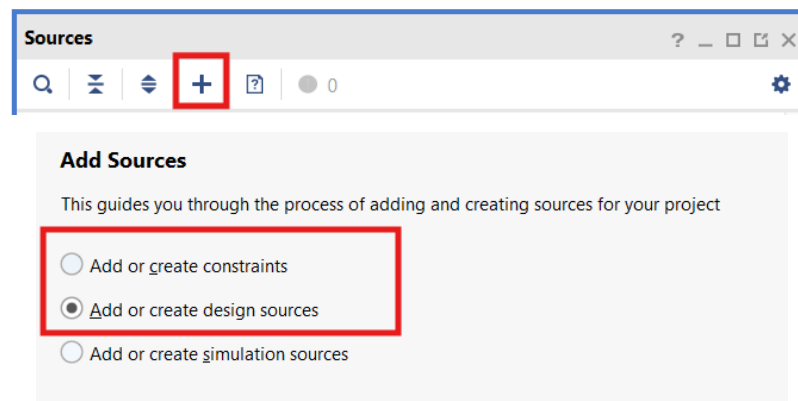
Error messages are essential for debugging in the design, typically you will get an error message when your design violates the compiler hard. In this part, you will learn how to read error messages.

Instructions

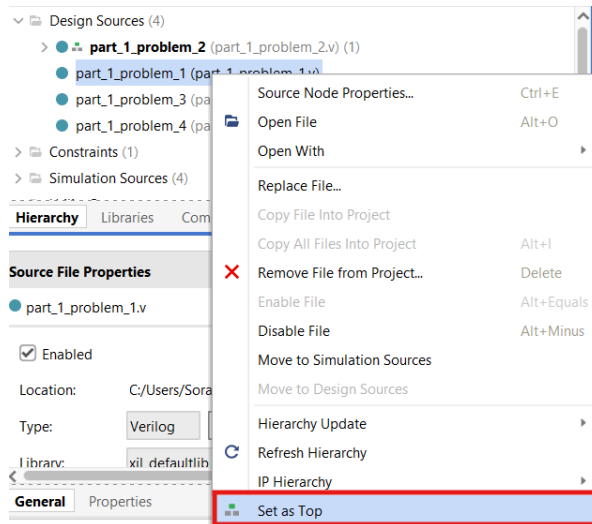
Clone the assignment from this repository:

<https://github.com/2110363-HW-SYN-LAB/lab3>

- a. Create a new Vivado project.
- b. Import each problem source file from the “part1” directory by clicking the “+” button in the source section.

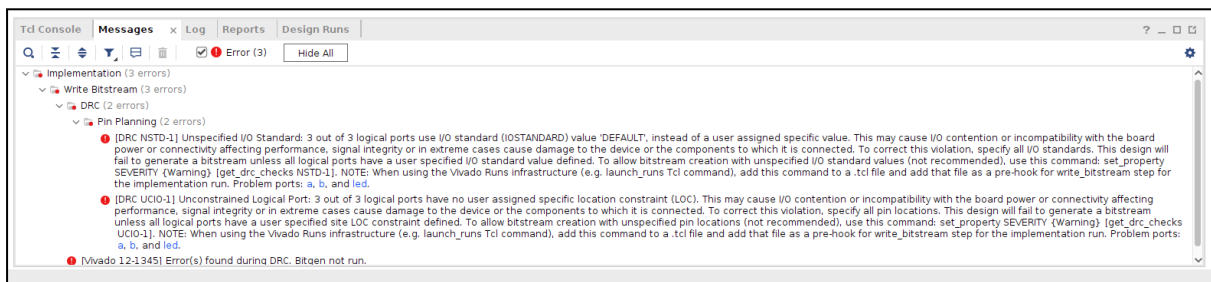


- Import *part_1_problem_1.v*, *part_1_problem_2.v*, *part_1_problem_3.v*, and *part_1_problem_4.v* as design sources.
 - Import *part_1_problem_3.xdc* as constraint.
- c. Generate bitstream for each problem.
 - i. Right click on file *part_1_problem_x* and select “Set as top”. After clicking, the file name should be bolded.



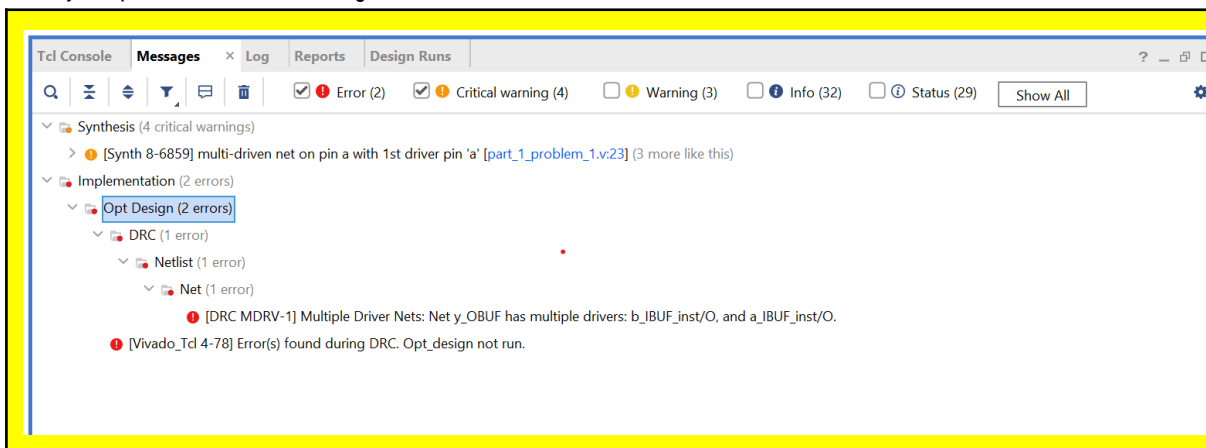
ii. Run “Generate Bitstream”.

d. Observe the error message.



e. Capture each error message.

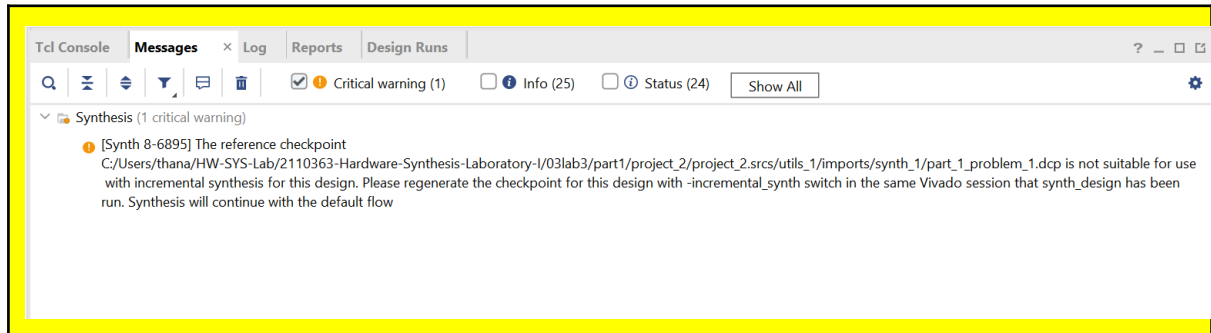
Paste your problem 1 error message



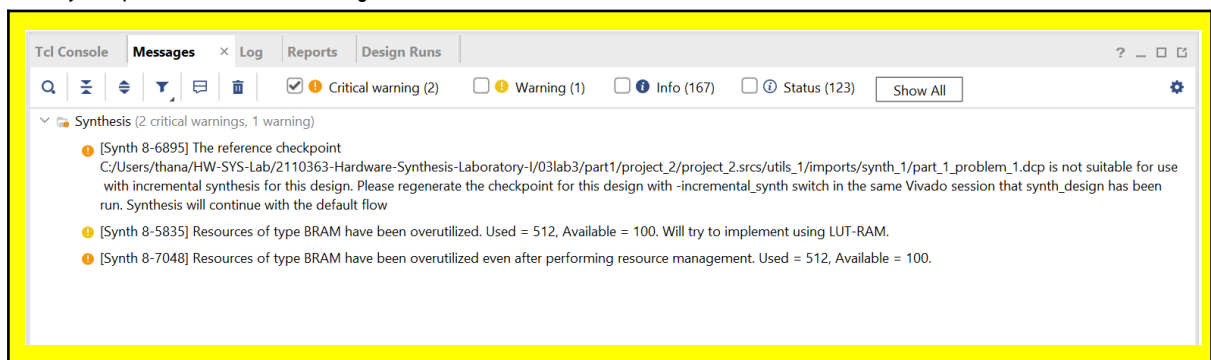
Paste your problem 2 error message



Paste your problem 3 error message



Paste your problem 4 error message



f. Answer the questions in MCV.

Part 2: Width Mismatching

Width mismatching is a common bug that occurs when you mismatch data width in your design, this can lead to unpredictable behavior of your design. Luckily we can detect this bug by using the linting.

Project Overview

In this project, you are given 3 modules:

1. `top_module`: the top module for this part
2. `producer`: the module that produce data
3. `consumer`: the module that consume data

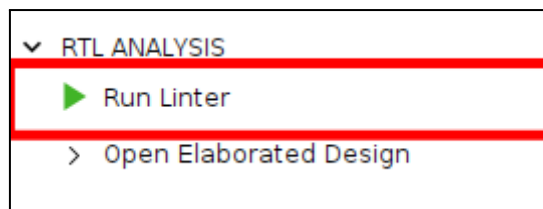
We will connect the module in 3 ways: fully connected, partial connected and not connected. Your task is to run linting and capture the output of each module connection type.

Step 1: Create a new Vivado project

Create a new Vivado project and import 3 Verilog files in “part2” directory as design sources.

Step 2: Run Linting

- A. Click the “Run Linter”.

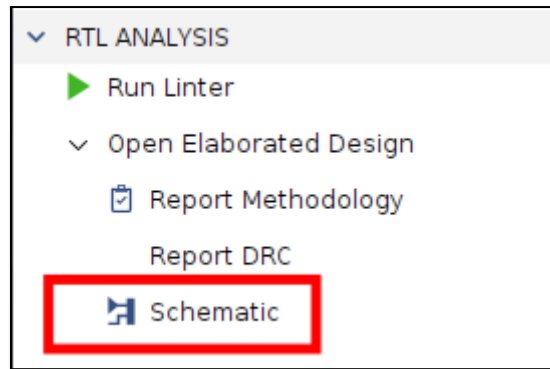


Step 3: Open the Schematic

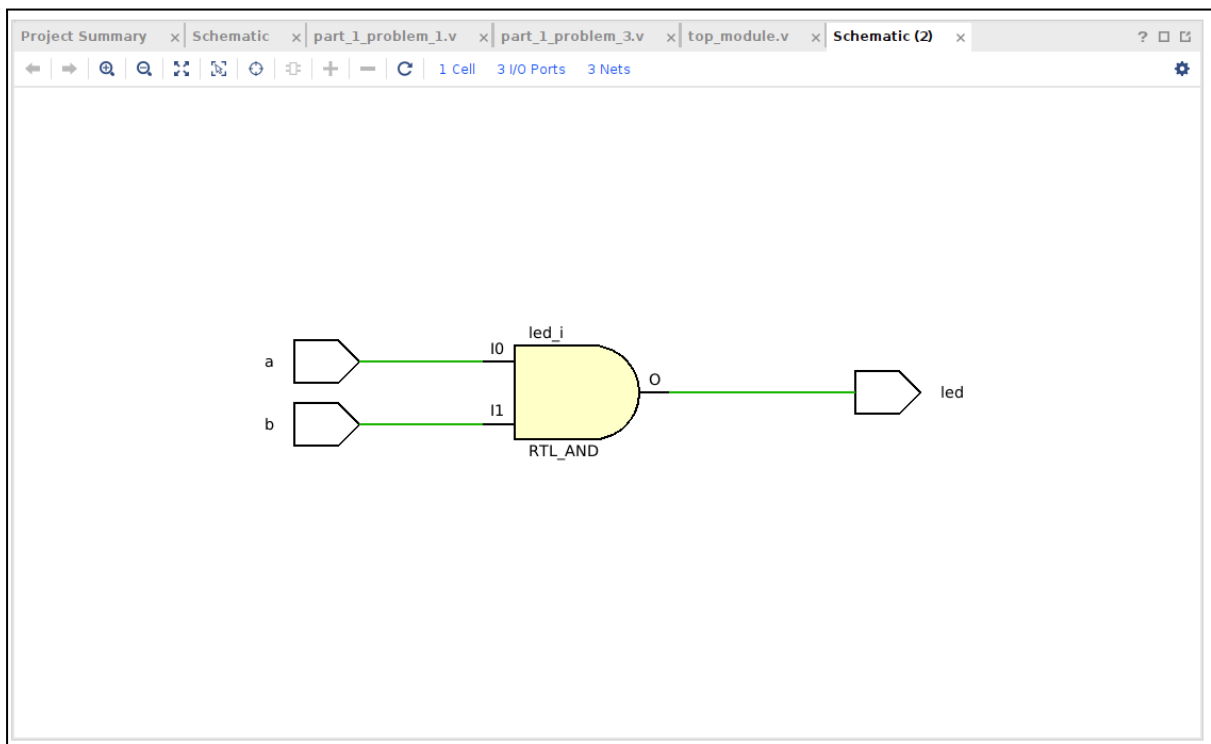
- A. Click “Open Elaborated Design”.



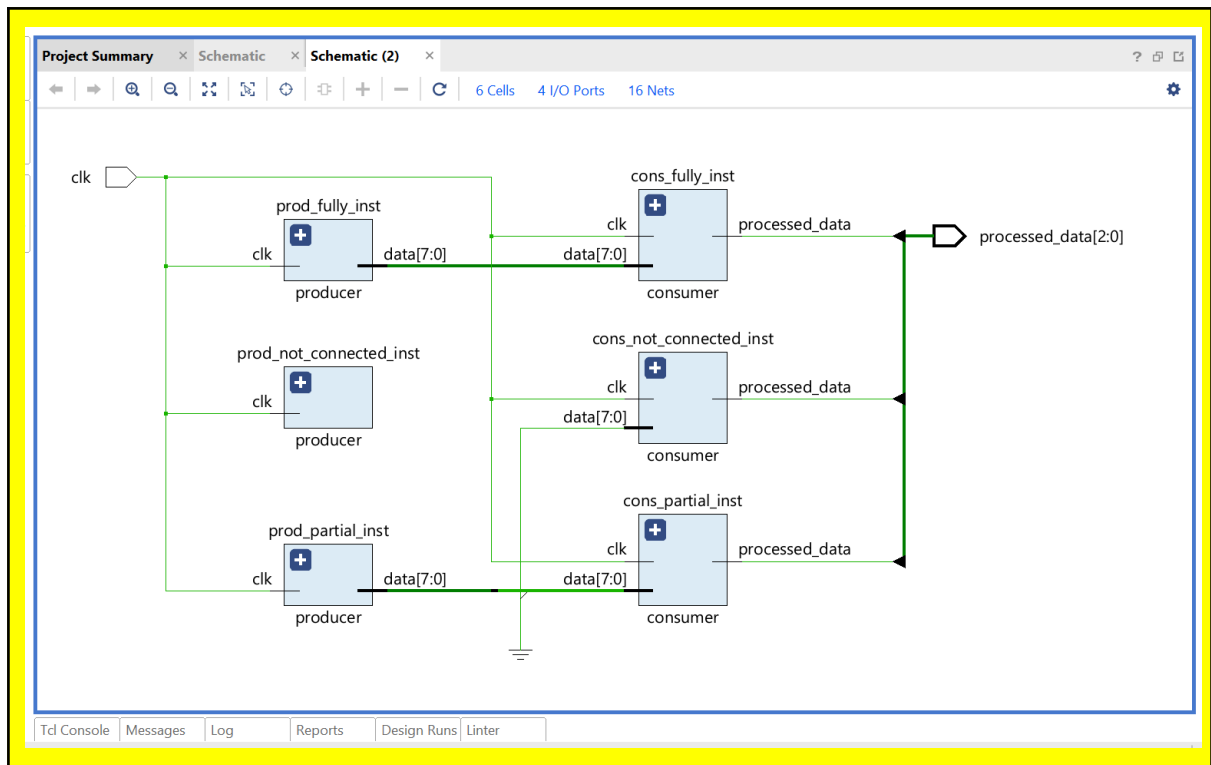
- B. Click “Schematic”.



- C. Your screen will show the schematic of your design. Capture your schematic and paste on the box below. This image below is the example of the schematic (not for this problem).



Capture your schematic and paste your schematic here



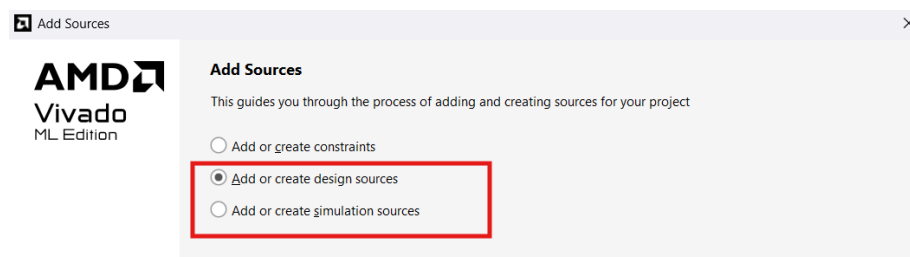
Part 3: Debugging using Simulation

Digital simulation is a crucial tool for debugging digital design. It is usually faster to run the digital simulation rather than generate a bitstream directly.

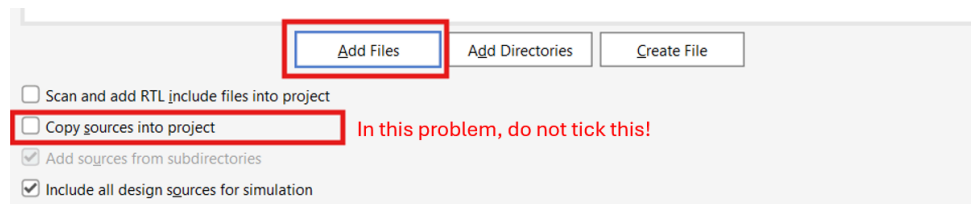
Instructions

- a. In each problem, create a new Vivado project and import Verilog files.
 - For problem 1, import *full_adder_4_bits.v* as a design source and *full_adder_4_bits_tb.v* as a simulation source.
 - For problem 2, import *fsm.v* as a design source and *fsm_tb.v* as a simulation source.

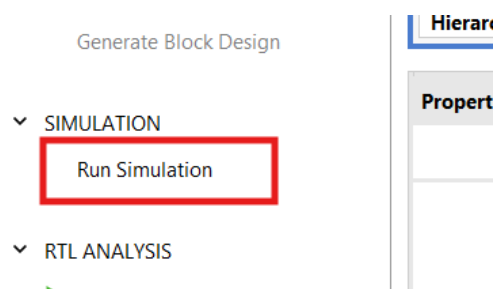
You can select if the file is a design or simulation source while you are adding new files.



You can import by clicking “Add Files”. However, in this problem, **do not** tick “Copy sources into project” or the grader will not work.

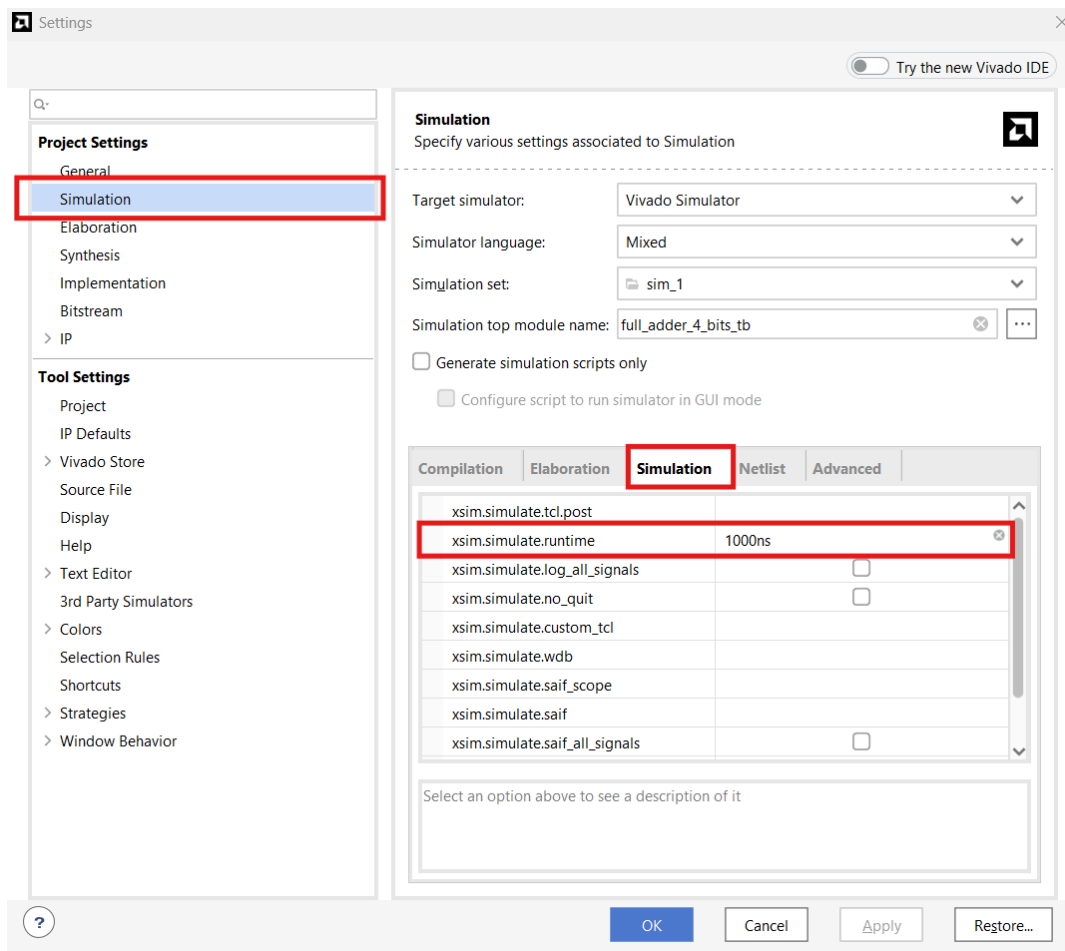


- b. Your task is to edit Verilog **simulation/testbench** files (files ended with *_tb.v*) in each problem.
- c. After editing codes, verify your testbenches by simulating inside Vivado and capture the waveform. You can simulate by clicking “Run Simulation”.



- d. To check if your testbench simulates correctly, run *tb.py* inside each problem’s directory.

Note: The simulator in Vivado will simulate for 1000ns by default. If your waveform is cut at 1000ns, you can change the simulation runtime in “Simulation Settings” (right click on the “Run Simulation” button).



Problem 1

You are given a 4-bit full adder module in `full_adder_4_bits.v`, which adds two 4-bit inputs and carry-in. The module will give outputs as sum and carry-out.

Inputs

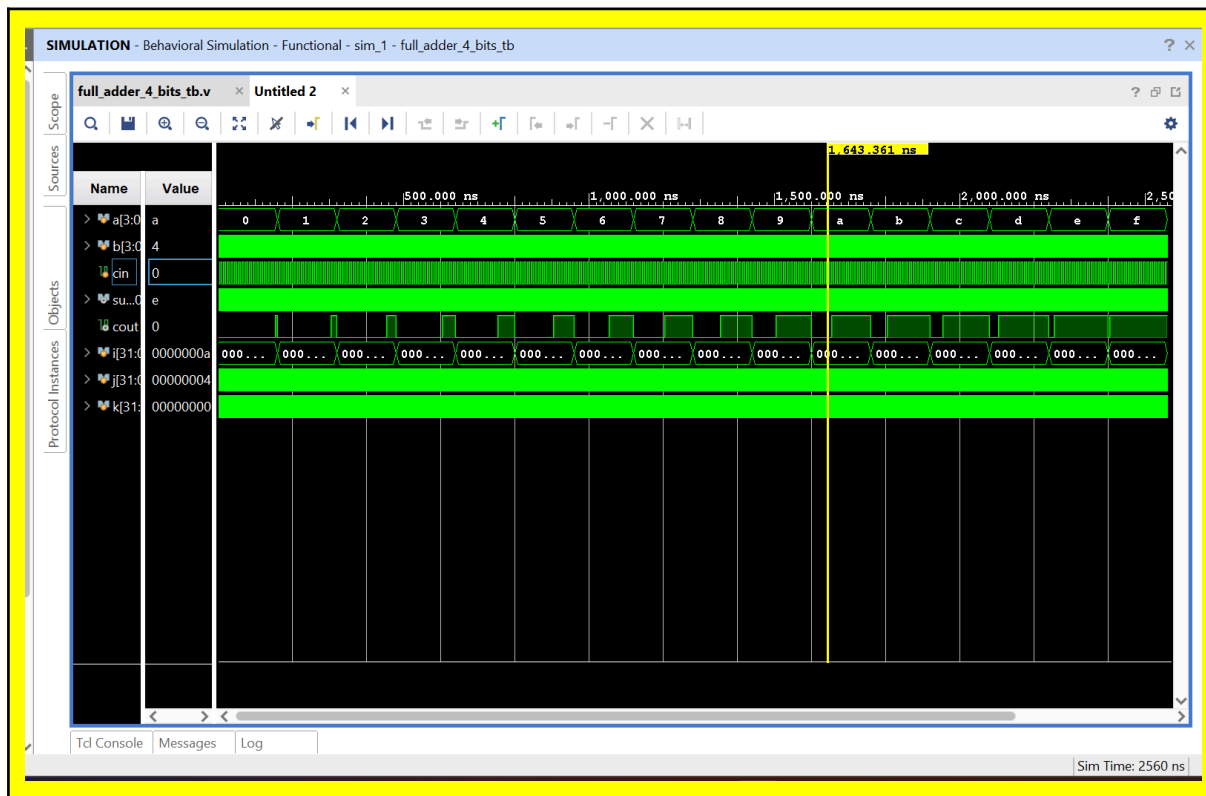
- `a`: 4-bit input of the first number.
- `b`: 4-bit input of the second number.
- `cin`: 1-bit input of the carry-in.

Outputs

- `sum`: 2-bit output of the result of addition.
- `cout`: 1-bit output of the carry-out of addition.

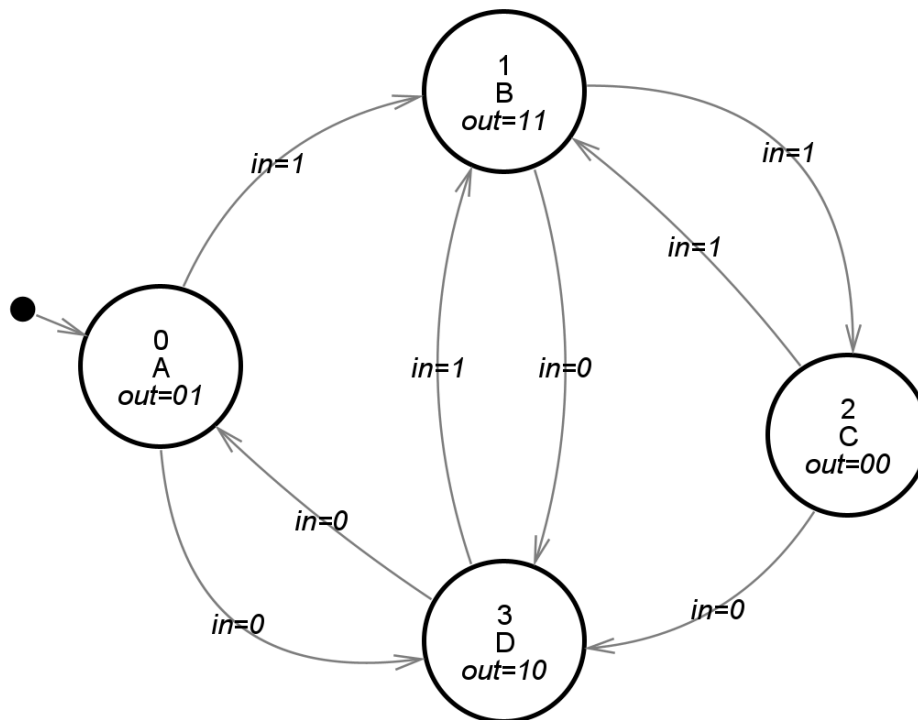
Write down a testbench for simulating the 4-bit full adder in `full_adder_4_bits_tb.v`. Your simulation must perform all combinations of inputs.

Paste your result waveform picture here



Problem 2

You are given a finite state machine in *fsm.v* with the state diagram below.



Inputs

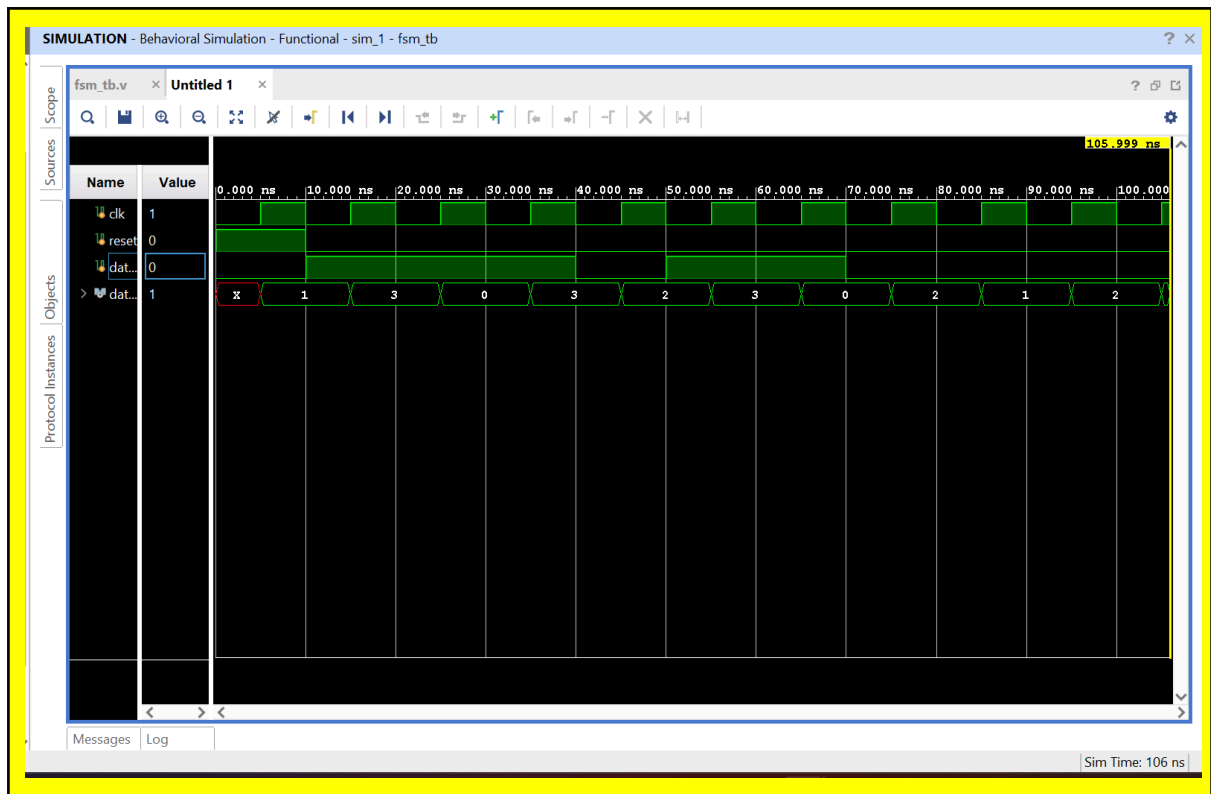
- *clk*: Clock signal that controls the finite state machine
- *rst*: Synchronous reset signal (active high) used to reset the machine
- *data_in*: 1-bit input signal that determines state transitions

Outputs

- *data_out*: 2-bit output signal determined solely by the current state

Write down a testbench for simulating this finite state machine in *fsm_tb.v*. Your simulation must have all possible state transitions. Also, your simulation must reset the FSM first before simulating other state transitions.

Paste your result waveform picture here



**** Checkpoint: Run tb.py in each problem and call TA to check your work ****

Part 4: Debugging using ILA and VIO

**** In this part, work in pair with one of your friend. Make sure there is at least 1 person in your pair who uses Windows or Linux. ILA and VIO only work natively in Windows and Linux. ****

ILA and VIO are also other tools that we can use to debug the design. In this part you will learn how to use ILA and VIO in any design.

Project Overview

In this project you are given a data generator module. This module will produce data once it receives a start signal and you can configure how much data you want it to generate. We will use the VIO to configure and start the data transfer and view the output data via the ILA.

Step 1: Setup project

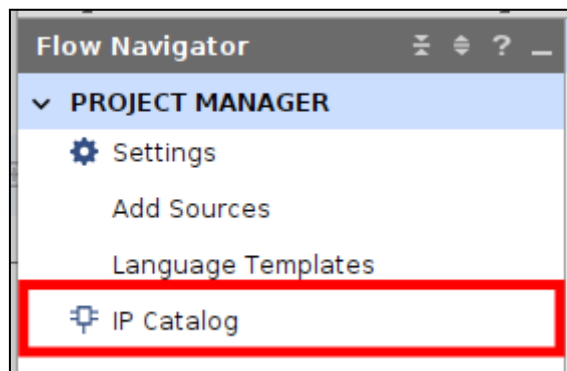
Create new Vivado project and import the following files:

- Import *data_generator.v* and *design_under_test_top.v* as design sources.
- Import *part_4_constraint.xdc* as constraint.

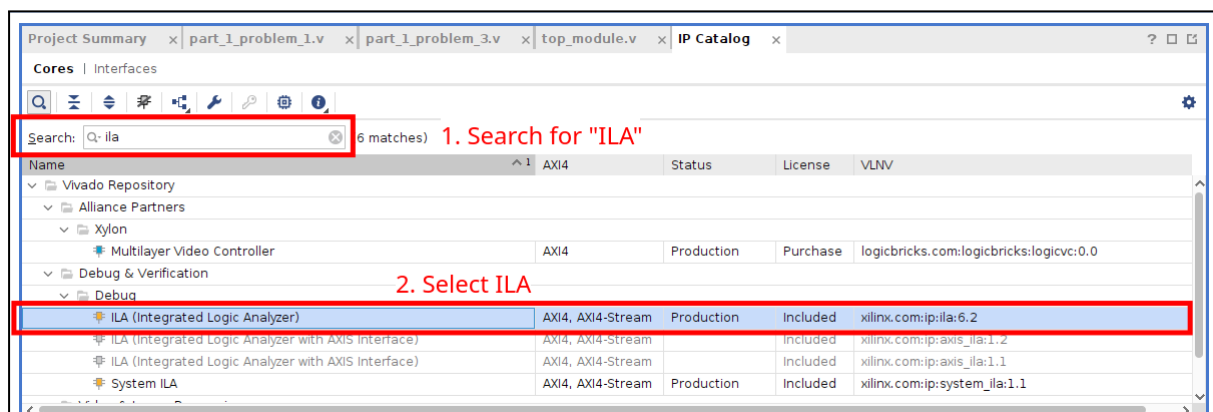
Step 2: Create the ILA module

Now, we will create an ILA instance to probe the signal inside the design.

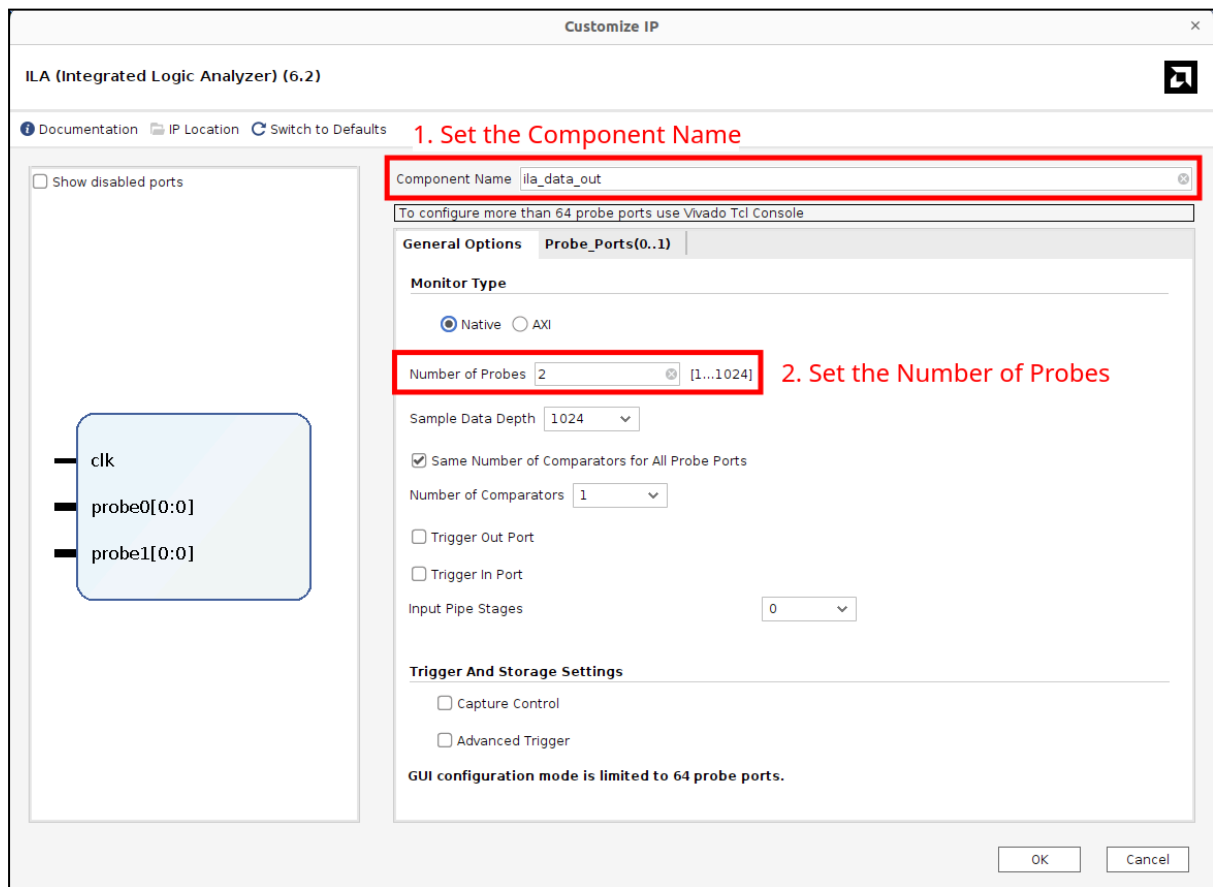
- A. Open the “IP Catalog”.



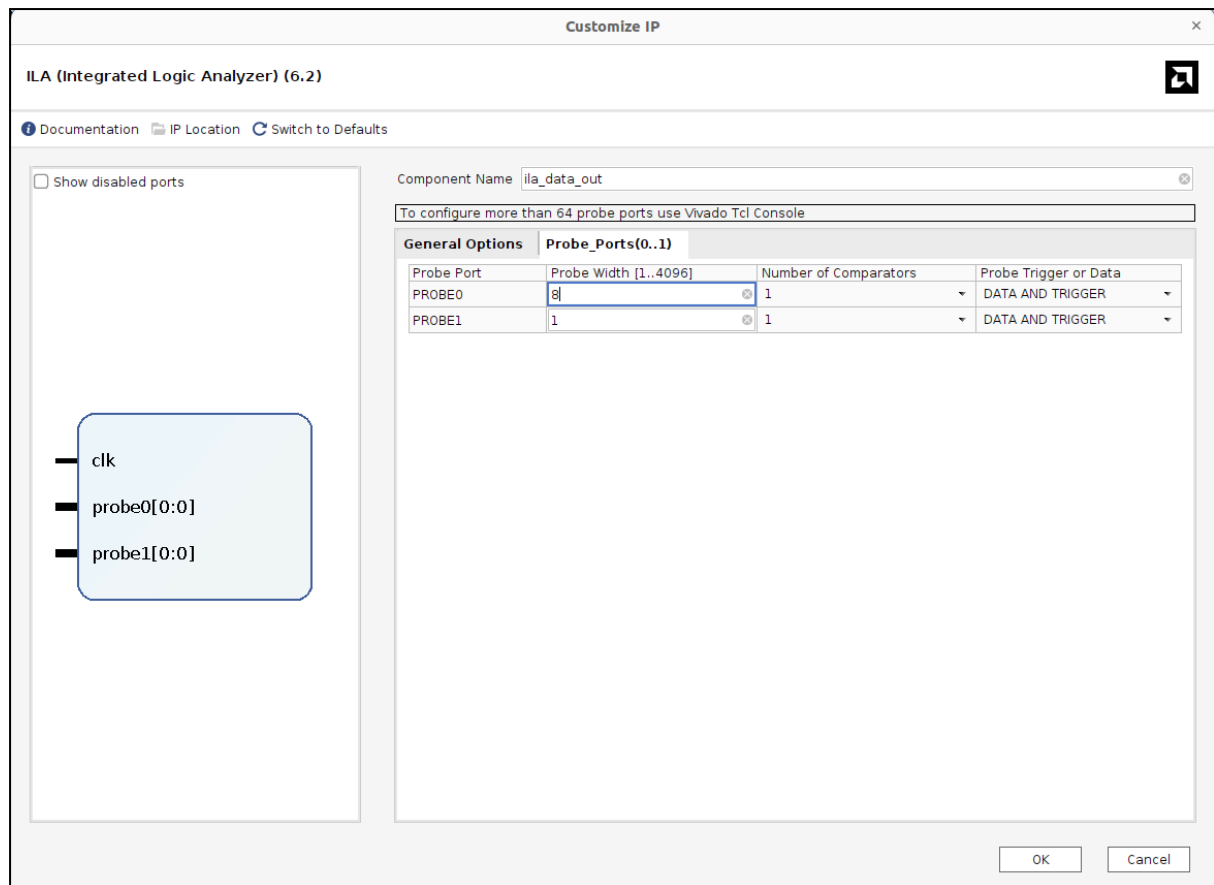
- B. Search for “ILA” and select the ILA.



- C. Configure the ILA.
- Set the component name of the ILA to “ila_data_out”.
 - Set the number of probes to 2.



- Direct to the “Probe_Ports(0...1)” tab and set the PROBE0’s data width to 8 bit wide, and then click “OK” to create the ILA. You may be prompted to click “Generate” button after clicking the “OK” button.



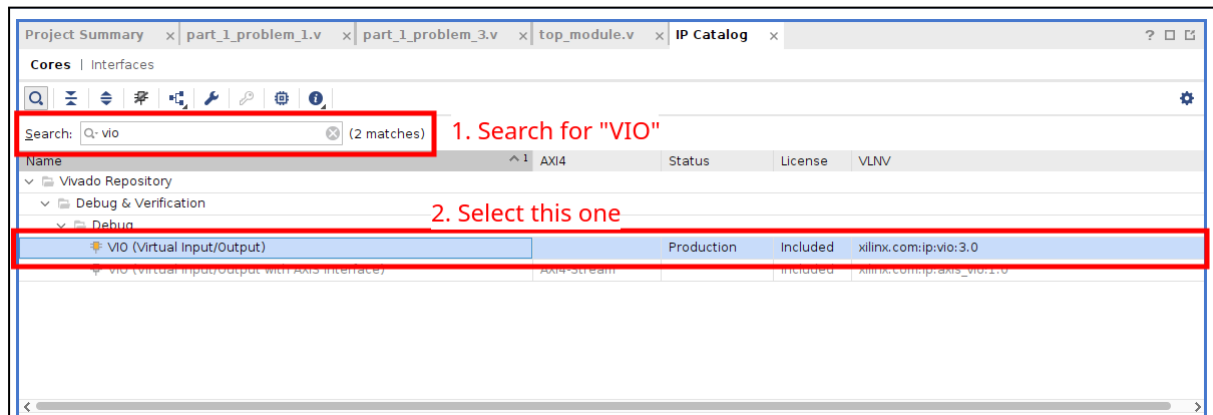
- D. Now, we will connect probes of ILA to our design. Inside *design_under_test_top.v*, fill the following code to create an ILA instance and connect the probes.

```
ila_data_out ila_inst (
    .clk      (clk),
    .probe0   (data_out),
    .probe1   (data_valid)
);
```

Step 3: Create the VIO module

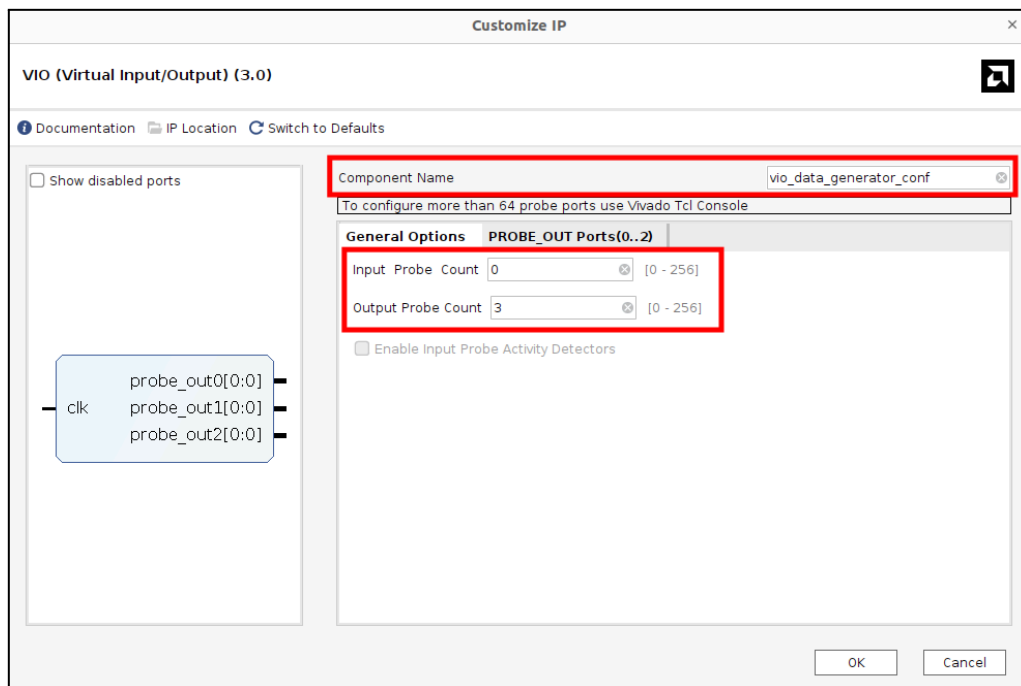
We will create a VIO instance to drive signals to our design.

- A. Open the IP catalog and search for VIO.

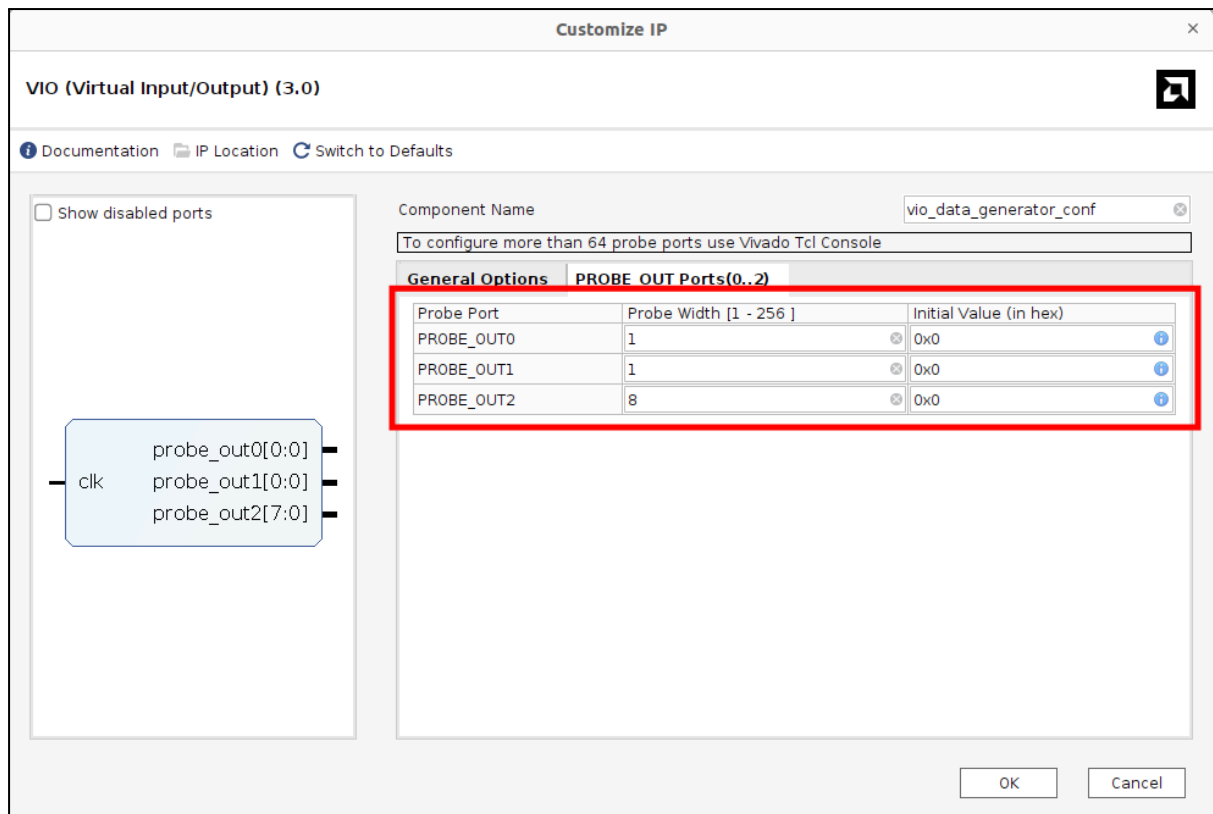


- B. Configure the VIO.

- Set the component name of the ilar to "vio_data_generator_conf".
- Set the number of input probes to 0.
- Set the number of output probes to 3.



- Direct to the "PROBE_OUT Ports(0..2)" tab and set the PROBE_OUT2 data width to 8 bit wide., and then click "OK" to generate VIO. You may be prompted to click "Generate" button after clicking the "OK" button.



- C. Now, we will connect VIO to our design. Inside *design_under_test_top.v*, fill the following code to create a VIO instance.

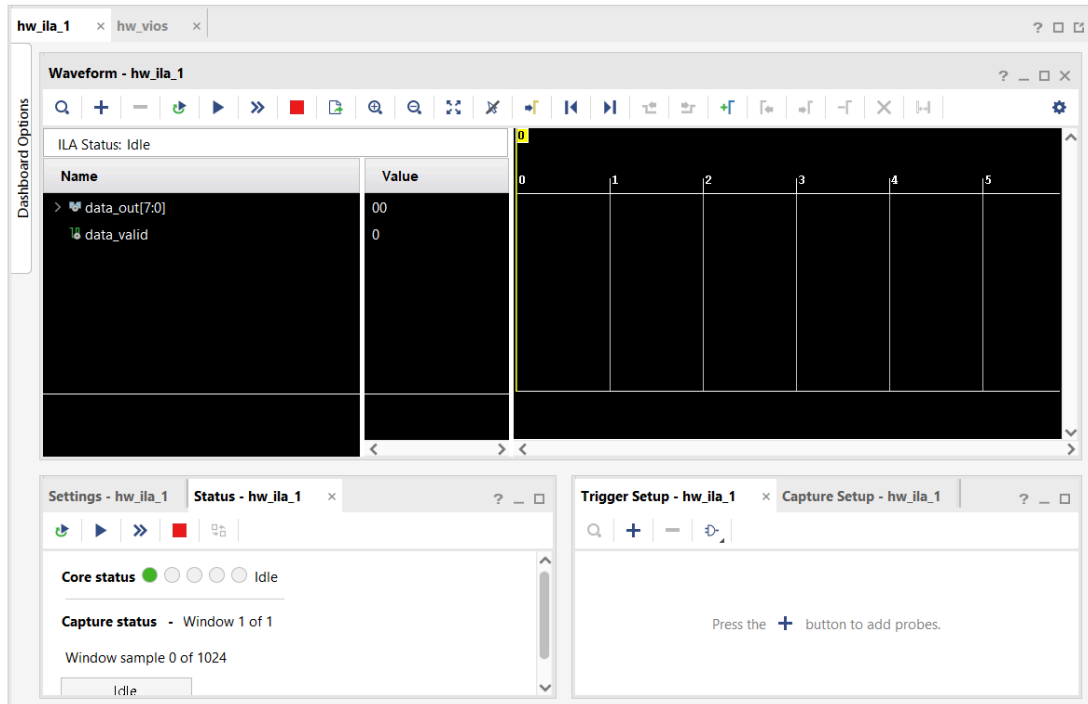
```
vio_data_generator_conf vio_inst (
    .clk          (clk) ,
    .probe_out0 (start) ,
    .probe_out1 (rst) ,
    .probe_out2 (data_length)
);
```

Step 4: Generate Bitstream

Click "Generate Bitstream" at the left of the program. This will launch synthesis and implementation automatically before generating bitstream.

Step 5: Program the Board

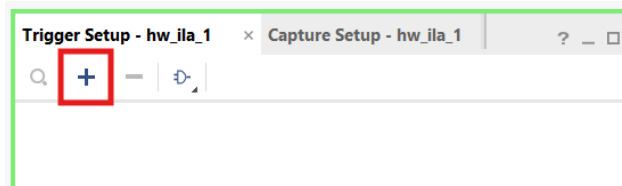
Connect your board to your computer and program the board. After you program the board, the dashboard should appear like the image below.



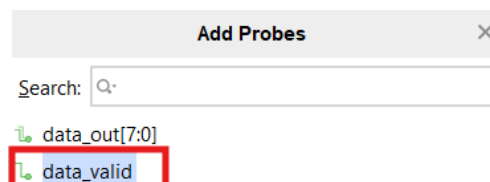
Step 6: Setup the ILA

We will let ILA trigger when the `data_valid` is high and `data_out` is `0x04`.

- A. At the “Trigger Setup - hw_ila_1” tab in the bottom right of the dashboard, click the “+” button to add probes.



- B. Select “data_valid”.



- C. We will let ILA trigger when `data_valid` is 1. Choose the operator to “==”, radix to binary, and value to “1”.

Name	Operator	Radix	Value	Port	Comparator Usage
data_valid	==	[B]	1	probe1[0]	1 of 1

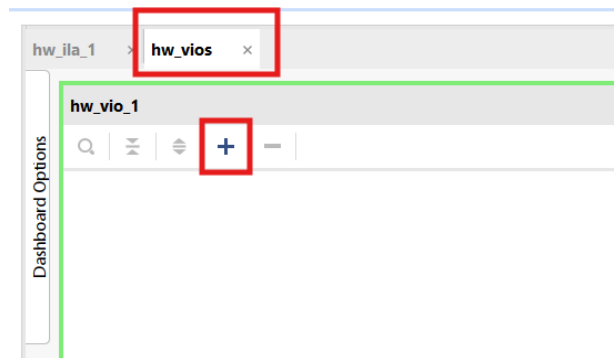
- D. Create another probe by clicking the “+” button again and selecting “data_out[7:0]”. We want to trigger when data_out is 0x04, choose the operator to “==”, radix to hexadecimal, and value to “04”.

Name	Operator	Radix	Value	Port	Comparator Usage
data_valid	==	[B]	1	probe1[0]	1 of 1
data_out[7:0]	==	[H]	04	probe0[7:0]	1 of 1

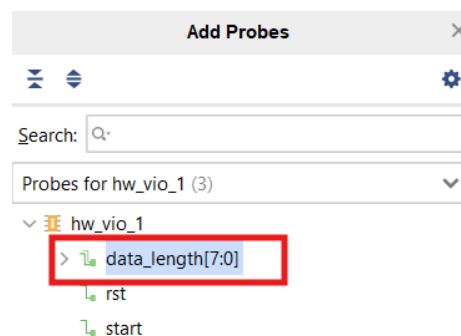
Step 7: Setup the VIO

We want to generate signals with data_length value to 0x10 and start value to 1.

- A. Go to “hw_vios” tab and click “+” button to add a probe.



- B. Select “data_length[7:0]”.



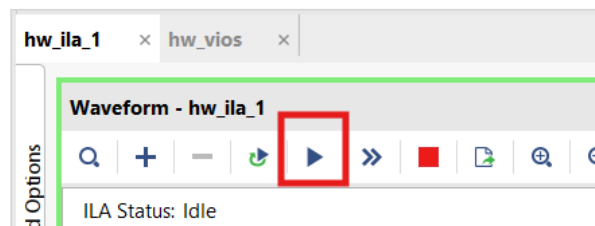
- C. Set the value of data_length[7:0] to 10 in hexadecimal.
- D. Now, create another probe for the start signal, click the “+” button again and select “start”. Then set the value of start to 1 in binary.

Name	Value	Activity	Direction	VIO
> data_length[7:0]	[H] 10		Output	hw_vio_1
start	[B] 1		Output	hw_vio_1

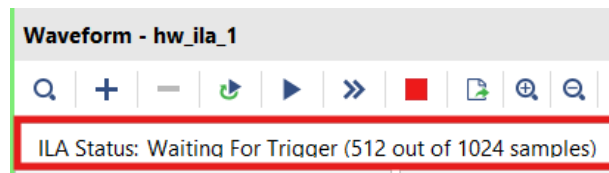
Step 8: Running

Now, we will test our setup, the ILA should trigger when the data is generated.

- A. Go back to “hw_ila_1” tab and click “Run” button to run the ILA.



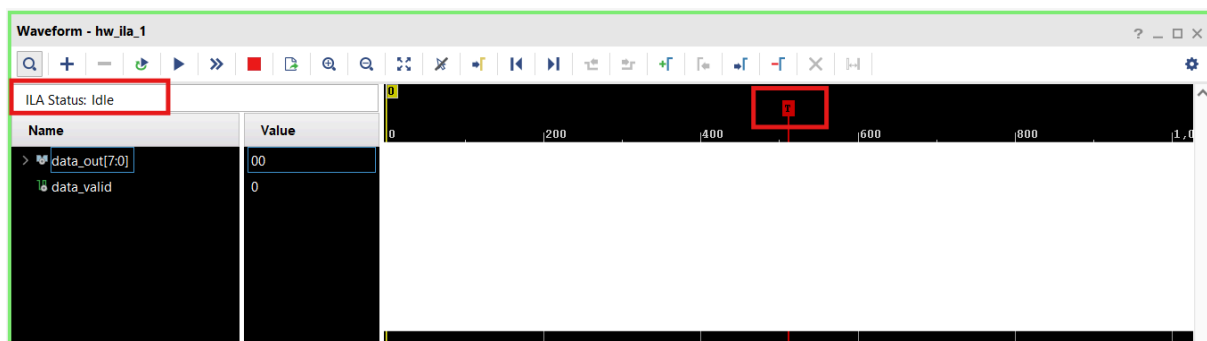
The status of the ILA should be “Waiting For Trigger”.



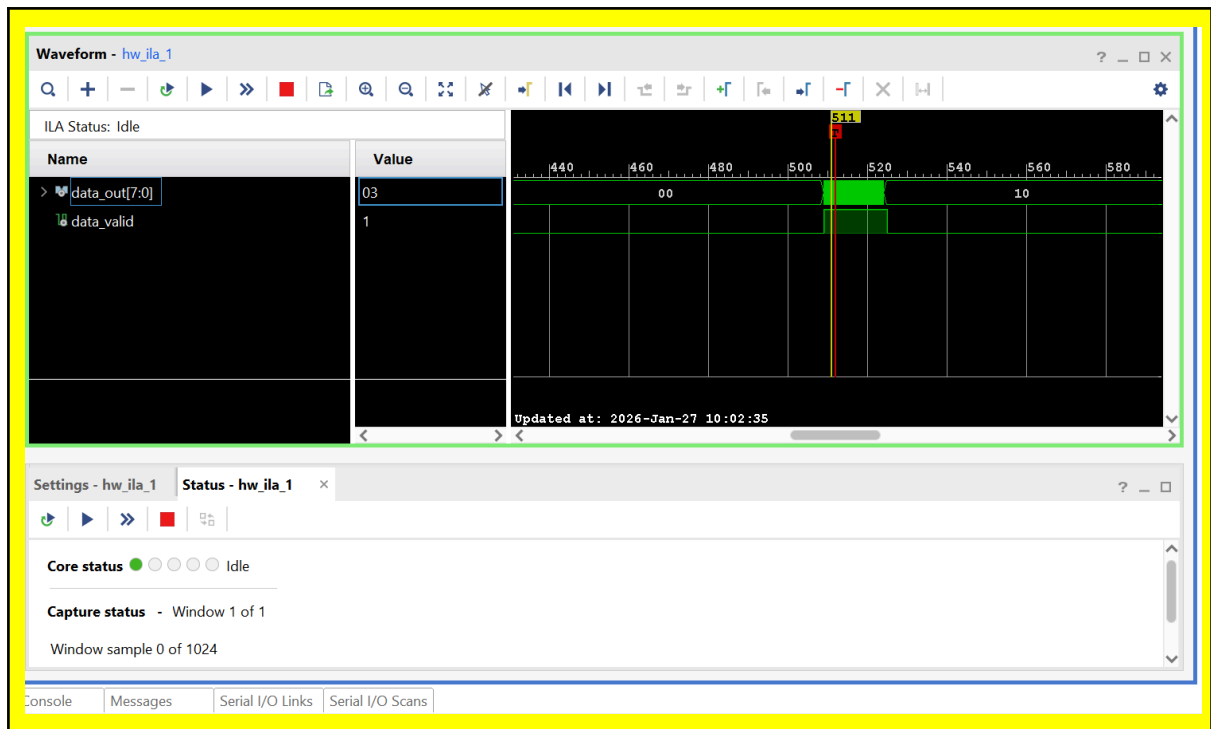
- B. Go back to “hw_vios” tab and switch start value from 1 to 0.

Name	Value	Activity	Direction
> data_length[7:0]	[H] 10		Output
start	[B] 0		Output

- C. Go back to “hw_ila_1” tab, you will see that the ILA will be triggered. Check if your ILA is triggered by viewing status of ILA (it should come back to “Idle”) and the waveform should appear with the “T” marker. Capture the waveform and paste in the box below.



Paste your result waveform picture from ILA here



**** Checkpoint: Call TA to check your work, export code of part 3 as zip file, export this document as PDF, and submit in MCV. ****